

SQL ASSIGNMENT

- 23PA1A45D1
- Y Sathvik

```
-- =====
-- QUERY 1
-- =====

SELECT e.emp_name, d.dept_name, e.salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
/*
OUTPUT
+-----+-----+-----+
| emp_name | dept_name | salary |
+-----+-----+-----+
| John     | Engineering | 120000.0 |
| Mike     | Engineering | 95000.0 |
| David    | Engineering | 90000.0 |
| Sarah    | Data        | 140000.0 |
| Priya    | Data        | 110000.0 |
| Ravi     | Data        | 105000.0 |
| Anil     | Data        | 85000.0 |
| Emma     | Finance     | 125000.0 |
| Robert   | Finance     | 85000.0 |
| Sophia   | HR          | 95000.0 |
| Chris    | HR          | 65000.0 |
| James    | Sales       | 130000.0 |
| Linda    | Sales       | 100000.0 |
| Kevin    | Sales       | 95000.0 |
| Nancy    | Sales       | 75000.0 |
+-----+-----+-----+
*/

-- =====
-- QUERY 2
-- =====

SELECT d.dept_name, COUNT(e.emp_id) AS emp_count
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
/*
OUTPUT
+-----+-----+
| dept_name | emp_count |
+-----+-----+
| Engineering | 3 |
| Data        | 4 |
| Finance     | 2 |
| HR          | 2 |
| Sales       | 4 |
+-----+-----+
*/--

-- =====
-- QUERY 3
-- =====

SELECT e.emp_name AS employee_name,
       m.emp_name AS manager_name FROM employees
e LEFT JOIN employees m ON e.manager_id =
m.emp_id;
/*
OUTPUT
+-----+-----+
| employee_name | manager_name |
+-----+-----+
| John          | NULL         |
| Mike          | John         |
+-----+-----+
```

David	John	
Sarah	NULL	
Priya	Sarah	
Ravi	Sarah	
Anil	Priya	
Emma	NULL	
Robert	Emma	
Sophia	NULL	
Chris	Sophia	
James	NULL	
Linda	James	
Kevin	James	
Nancy	Linda	

*/

-- QUERY 4

```
SELECT d.dept_id, d.dept_name, SUM(e.salary) AS sum_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name;
```

/*

OUTPUT

dept_id	dept_name	sum_salary	
1	Engineering	305000.0	
2	Data	440000.0	
3	Finance	210000.0	
4	HR	160000.0	
5	Sales	400000.0	

*/

-- QUERY 5

```
SELECT d.dept_id, d.dept_name, AVG(e.salary) AS avg_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY avg_salary DESC;
```

/*

OUTPUT

dept_id	dept_name	avg_salary	
2	Data	110000.0	
3	Finance	105000.0	
1	Engineering	101666.666667	
5	Sales	100000.0	
4	HR	80000.0	

*/

-- QUERY 6

-- Department with Highest Average Salary

```
SELECT d.dept_id, d.dept_name, AVG(e.salary) AS avg_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY avg_salary DESC
LIMIT 1;
```

/*

OUTPUT

```

+-----+-----+-----+
| dept_id | dept_name | avg_salary |
+-----+-----+-----+
| 2       | Data      | 110000.0   | +-----+-----+-----+
----+
*/ --

=====
-- QUERY 7
-- Employees earning more than company average salary --
=====

SELECT e.emp_name, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
);
/*
OUTPUT
+-----+-----+
| emp_name | salary |
+-----+-----+
| John     | 120000.0 |
| Sarah    | 140000.0 |
| Priya    | 110000.0 |
| Ravi     | 105000.0 |
| Emma     | 125000.0 |
| James    | 130000.0 |
+-----+-----+
*/

-- =====
-- QUERY 8
-- Employees earning more than department average salary
-- =====

SELECT e.emp_name, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.dept_id = e.dept_id
);
/*
OUTPUT
+-----+-----+
| emp_name | salary |
+-----+-----+
| John     | 120000.0 |
| Sarah    | 140000.0 |
| Emma     | 125000.0 |
| Sophia   | 95000.0  |
| James    | 130000.0 |
+-----+-----+
*/

-- =====
-- QUERY 9
-- Second Highest Salary
-- =====

SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
/*
OUTPUT
+-----+
| second_highest_salary |
+-----+

```

```

+-----+
| 130000.0 |
+-----+
*/ --
=====
-- QUERY 10
-- Employees who earn maximum salary in their department
-- =====
SELECT e.emp_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary = (
    SELECT MAX(e2.salary)
    FROM employees e2    WHERE
e2.dept_id = e.dept_id
);
/*
OUTPUT
+-----+-----+-----+
| emp_name | dept_id | salary |
+-----+-----+-----+
| John     | 1       | 120000.0 |
| Sarah    | 2       | 140000.0 |
| Emma     | 3       | 125000.0 |
| Sophia   | 4       | 95000.0  |
| James    | 5       | 130000.0 |
+-----+-----+-----+
*/
=====
-- QUERY 11
-- Rank employees based on salary within each department
-- =====
SELECT e.emp_id,
       e.emp_name,
       e.dept_id,
       e.salary,
RANK() OVER (
    PARTITION BY e.dept_id
    ORDER BY e.salary DESC
) AS salary_rank
FROM employees e;
/*
OUTPUT
+-----+-----+-----+-----+-----+
| emp_id | emp_name | dept_id | salary | salary_rank |
+-----+-----+-----+-----+-----+
| 101    | John     | 1       | 120000.0 | 1           |
| 102    | Mike     | 1       | 95000.0  | 2           |
| 103    | David    | 1       | 90000.0  | 3           |
| 104    | Sarah    | 2       | 140000.0 | 1           |
| 105    | Priya    | 2       | 110000.0 | 2           |
| 106    | Ravi     | 2       | 105000.0 | 3           |
| 107    | Anil     | 2       | 85000.0  | 4           |
| 108    | Emma     | 3       | 125000.0 | 1           |
| 109    | Robert   | 3       | 85000.0  | 2           |
| 110    | Sophia   | 4       | 95000.0  | 1           |
| 111    | Chris    | 4       | 65000.0  | 2           |
| 112    | James    | 5       | 130000.0 | 1           |
| 113    | Linda    | 5       | 100000.0 | 2           |
| 114    | Kevin    | 5       | 95000.0  | 3           |
| 115    | Nancy    | 5       | 75000.0  | 4           |
+-----+-----+-----+-----+-----+
*/
=====
-- QUERY 12
-- Top 3 highest paid employees in each department
-- =====

```

```

SELECT *
FROM (
    SELECT
    e.emp_id,
        e.emp_name,
        e.dept_id,
        e.salary,
    DENSE_RANK() OVER (
        PARTITION BY e.dept_id
        ORDER BY e.salary DESC
    ) AS salary_rank
    FROM employees e
) t
WHERE salary_rank <= 3;
/*
OUTPUT
+-----+-----+-----+-----+-----+
| emp_id | emp_name | dept_id | salary   | salary_rank |
+-----+-----+-----+-----+-----+
| 101    | John    | 1       | 120000.0 | 1           |
| 102    | Mike    | 1       | 95000.0  | 2           |
| 103    | David   | 1       | 90000.0  | 3           |
| 104    | Sarah   | 2       | 140000.0 | 1           |
| 105    | Priya   | 2       | 110000.0 | 2           |
| 106    | Ravi    | 2       | 105000.0 | 3           |
| 108    | Emma    | 3       | 125000.0 | 1           |
| 109    | Robert  | 3       | 85000.0  | 2           |
| 110    | Sophia  | 4       | 95000.0  | 1           |
| 111    | Chris   | 4       | 65000.0  | 2           |
| 112    | James   | 5       | 130000.0 | 1           |
| 113    | Linda   | 5       | 100000.0 | 2           |
| 114    | Kevin   | 5       | 95000.0  | 3           |
+-----+-----+-----+-----+-----+
*/

-- =====
-- QUERY 13
-- Running Total Salary Ordered by Hire Date
-- =====
SELECT
e.emp_id,
    e.emp_name,
    e.hire_date,
    e.salary,
SUM(e.salary) OVER (
    ORDER BY e.hire_date
) AS running_total_salary
FROM employees e
ORDER BY e.hire_date;
/*
OUTPUT
+-----+-----+-----+-----+-----+
| emp_id | emp_name | hire_date | salary   | running_total_salary |
+-----+-----+-----+-----+-----+
| 108    | Emma    | 2016-11-18 | 125000.0 | 125000.0             |
| 104    | Sarah   | 2017-08-25 | 140000.0 | 265000.0             |
| 101    | John    | 2018-01-15 | 120000.0 | 385000.0             |
| 112    | James   | 2018-12-05 | 130000.0 | 515000.0             |
| 102    | Mike    | 2019-03-10 | 95000.0  | 610000.0             |
| 113    | Linda   | 2019-10-10 | 100000.0 | 710000.0             |
| 105    | Priya   | 2020-02-14 | 110000.0 | 820000.0             |
| 109    | Robert  | 2020-04-21 | 85000.0  | 905000.0             |
| 103    | David   | 2021-06-20 | 90000.0  | 995000.0             |
| 110    | Sophia  | 2021-09-11 | 95000.0  | 1090000.0            |
| 106    | Ravi    | 2022-01-10 | 105000.0 | 1195000.0            |
| 114    | Kevin   | 2022-03-15 | 95000.0  | 1290000.0            |
| 107    | Anil    | 2023-05-12 | 85000.0  | 1375000.0            |
| 111    | Chris   | 2024-01-15 | 65000.0  | 1440000.0            |
| 115    | Nancy   | 2024-06-01 | 75000.0  | 1515000.0            |
+-----+-----+-----+-----+-----+

```

```

+-----+-----+-----+-----+-----+
*/
-- =====
-- QUERY 14
-- Previous Employee Salary using LAG()
-- =====

SELECT e.emp_id,
       e.emp_name,
       e.salary,
       LAG(e.salary) OVER (
           ORDER BY e.hire_date
       ) AS previous_salary
FROM employees e
ORDER BY e.hire_date;
/*
OUTPUT
+-----+-----+-----+-----+-----+
| emp_id | emp_name | salary   | previous_salary |
+-----+-----+-----+-----+-----+
| 108    | Emma    | 125000.0 | NULL           |
| 104    | Sarah   | 140000.0 | 125000.0       |
| 101    | John    | 120000.0 | 140000.0       |
| 112    | James   | 130000.0 | 120000.0       |
| 102    | Mike    | 95000.0  | 130000.0       |
| 113    | Linda   | 100000.0 | 95000.0        |
| 105    | Priya   | 110000.0 | 100000.0       |
| 109    | Robert  | 85000.0  | 110000.0       |
| 103    | David   | 90000.0  | 85000.0        |
| 110    | Sophia  | 95000.0  | 90000.0        |
| 106    | Ravi    | 105000.0 | 95000.0        |
| 114    | Kevin   | 95000.0  | 105000.0       |
| 107    | Anil    | 85000.0  | 95000.0        |
| 111    | Chris   | 65000.0  | 85000.0        |
| 115    | Nancy   | 75000.0  | 65000.0        |
+-----+-----+-----+-----+-----+
*/ --
-- =====
-- QUERY 15
-- Customers who never placed an order
--
-- =====

SELECT c.customer_id,
       c.customer_name,
       c.city
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
/*
OUTPUT
+-----+-----+-----+-----+-----+
| customer_id | customer_name | city       |
+-----+-----+-----+-----+-----+
| 7           | Amazon        | Hyderabad |
+-----+-----+-----+-----+-----+
*/
-- =====
-- QUERY 16
-- Latest Order for Each Customer
-- =====

SELECT customer_id, order_id, order_date, amount
FROM (
    SELECT o.*,
           ROW_NUMBER() OVER (
               PARTITION BY o.customer_id
               ORDER BY o.order_date DESC
           )

```

```

        ) AS rn
    FROM orders o
) t
WHERE rn = 1;
/*
OUTPUT
+-----+-----+-----+-----+
| customer_id | order_id | order_date | amount |
+-----+-----+-----+-----+
| 1           | 1018     | 2024-09-20 | 50000.0 |
| 2           | 1020     | 2024-10-22 | 80000.0 |
| 3           | 1016     | 2024-08-15 | 65000.0 |
| 4           | 1014     | 2024-07-08 | 20000.0 |
| 5           | 1017     | 2024-09-01 | 45000.0 |
| 6           | 1019     | 2024-10-05 | 30000.0 |
+-----+-----+-----+-----+
*/

-- =====
-- QUERY 17
-- Total Sales Generated by Each Customer
-- =====
SELECT c.customer_name,
       SUM(o.amount) AS total_sales
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
/*
OUTPUT
+-----+-----+
| customer_name | total_sales |
+-----+-----+
| Amazon        | 210000.0    |
| Google        | 345000.0    |
| Microsoft     | 155000.0    |
| Netflix       | 35000.0     |
| Meta          | 155000.0    |
| Adobe         | 70000.0     |
+-----+-----+
*/

-- =====
-- QUERY 18
-- Customer Contribution Percentage to Overall Sales
-- =====
SELECT c.customer_name,
       SUM(o.amount) AS customer_sales,
       ROUND(
           SUM(o.amount) * 100.0 /
           (SELECT SUM(amount) FROM orders),
           2
       ) AS contribution_percentage
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
/*
OUTPUT
+-----+-----+-----+
| customer_name | customer_sales | contribution_percentage |
+-----+-----+-----+
| Amazon        | 210000.0       | 21.65                  |
| Google        | 345000.0       | 35.57                  |
| Microsoft     | 155000.0       | 15.98                  |
| Netflix       | 35000.0        | 3.61                   |
| Meta          | 155000.0       | 15.98                  |
| Adobe         | 70000.0        | 7.22                   |
+-----+-----+-----+

```

```

+-----+-----+-----+
*/
-- =====
-- QUERY 19
-- Identify Duplicate Customer Names
-- =====
SELECT c.customer_name,
       COUNT(*) AS duplicate_count
FROM customers c
GROUP BY c.customer_name
HAVING COUNT(*) > 1;
/*
OUTPUT
+-----+-----+
| customer_name | duplicate_count |
+-----+-----+
| Amazon        | 2               |
+-----+-----+
*/ --
=====
-- QUERY 20
-- Classify Employees into Salary Bands
-- =====
SELECT e.emp_name AS Employee_Name,
       e.salary AS Salary,
CASE
    WHEN e.salary < 80000 THEN 'Low'
    WHEN e.salary BETWEEN 80000 AND 120000 THEN 'Medium'
    ELSE 'High'
END AS Salary_Band
FROM employees e;
/*
OUTPUT
+-----+-----+-----+
| Employee_Name | Salary   | Salary_Band |
+-----+-----+-----+
| John          | 120000.0 | Medium      |
| Mike          | 95000.0  | Medium      |
| David         | 90000.0  | Medium      |
| Sarah         | 140000.0 | High        |
| Priya         | 110000.0 | Medium      |
| Ravi          | 105000.0 | Medium      |
| Anil          | 85000.0  | Medium      |
| Emma          | 125000.0 | High        |
| Robert        | 85000.0  | Medium      |
| Sophia        | 95000.0  | Medium      |
| Chris         | 65000.0  | Low         |
| James         | 130000.0 | High        |
| Linda         | 100000.0 | Medium      |
| Kevin         | 95000.0  | Medium      |
| Nancy         | 75000.0  | Low         |
+-----+-----+-----+
*/
-- =====
-- QUERY 21
-- Find Employees Having Same Salary
-- =====
SELECT e1.emp_name, e1.salary

FROM employees e1
JOIN employees e2
ON e1.salary = e2.salary
AND e1.emp_id <> e2.emp_id
ORDER BY e1.salary;
/*
OUTPUT
+-----+

```



```

| emp_name | salary |
+-----+-----+
| Robert   | 85000.0 |
| Anil     | 85000.0 |
| Kevin    | 95000.0 |
| Sophia   | 95000.0 |
| Kevin    | 95000.0 |
| Mike     | 95000.0 |
| Sophia   | 95000.0 |
| Mike     | 95000.0 |
+-----+-----+
*/

-- =====
-- QUERY 22
-- Customers Who Placed More Than 3 Orders
-- =====

SELECT o.customer_id,
       COUNT(*) AS order_count
FROM orders o
GROUP BY o.customer_id
HAVING COUNT(*) > 3;
/*
OUTPUT
+-----+-----+
| customer_id | order_count |
+-----+-----+
| 1           | 5           |
| 2           | 5           |
+-----+-----+
*/

-- =====
-- QUERY 23
-- Month With Highest Sales
-- =====

SELECT MONTH(order_date) AS order_month,
       SUM(amount) AS total_sales
FROM orders
GROUP BY MONTH(order_date)
ORDER BY total_sales DESC
LIMIT 1;
/*
OUTPUT
+-----+-----+
| order_month | total_sales |
+-----+-----+
| 4           | 130000.0   |
+-----+-----+
*/

-- =====
-- QUERY 24
-- Days Between Consecutive Orders For Each Customer
-- =====

SELECT o.customer_id,
       o.order_id,
       o.order_date,
       DATEDIFF(
         o.order_date,
         LAG(o.order_date)
         OVER (
           PARTITION BY o.customer_id
           ORDER BY o.order_date
         )
       ) AS days_between_orders
FROM orders o;
/*
OUTPUT

```

customer_id	order_id	order_date	days_between_orders
1	1001	2024-01-05	NULL
1	1003	2024-02-02	28
1	1007	2024-04-10	68
1	1013	2024-07-01	82
1	1018	2024-09-20	81
2	1002	2024-01-10	NULL
2	1005	2024-03-01	51
2	1011	2024-06-10	101
1015	2024-08-10	61	
2	1020	2024-10-22	73
3	1004	2024-02-15	NULL
3	1009	2024-05-05	80
3	1016	2024-08-15	102
4	1006	2024-03-18	NULL
4	1014	2024-07-08	112
5	1008	2024-04-25	NULL
5	1012	2024-06-15	51
5	1017	2024-09-01	78
6	1010	2024-05-20	NULL
6	1019	2024-10-05	138

```

*/
-- =====
-- QUERY 25
-- Departments Where No Employee Was Hired
-- In The Last 2 Years
-- =====
SELECT d.dept_id,
       d.dept_name
FROM departments d
WHERE d.dept_id NOT IN (
    SELECT DISTINCT e.dept_id
    FROM employees e
    WHERE e.hire_date >= DATE_SUB(CURDATE(), INTERVAL 2 YEAR)
);

```

```

/*
OUTPUT

```

dept_id	dept_name
1	Engineering
2	Data
3	Finance
4	HR
5	Sales

```

*/

```